```
package com.ravi.tree_map_demo;

import java.util.Iterator;
import java.util.Map.Entry;
import java.util.TreeMap;

public class TreeMapDemo2
{
    public static void main(String args[])
  {
      TreeMap<Object,Object> map = new TreeMap<>();
      map.put("one","1");
      map.put("two",null);
      map.put("three","3");
         map.put("four",4);


      displayMap(map);

    map.forEach((k, v) -> IO.println("Key = " + k + ", Value = " + v));

  }
  static void displayMap(TreeMap<Object,Object> map)
  {
     Iterator<Entry<Object, Object>> itr = map.entrySet().iterator();
     itr.forEachRemaining(IO::println);

  }
}
```

```
package com.ravi.tree_map_demo;

import java.util.Comparator;
import java.util.TreeMap;

record Product(Integer id, String name) implements Comparator<Product>
{
    @Override
    public int compare(Product p1, Product p2)
    {
        return Integer.compare(p1.id(), p2.id());
    }

}
public class TreeMapDemo3
{
    public static void main(String[] args)
    {
        TreeMap<Product, String> map = new TreeMap<>();
        map.put(new Product(222, "Mobile"), "Hyderabad");
        map.put(new Product(111, "Laptop"), "Chennai");
        map.put(new Product(333, "Camera"), "Pune");

      map.forEach((k,v)-> IO.println(k+" : "+v));

    }

}
```

java.lang.ClassCastException

```
package com.ravi.tree_map_demo;

import java.util.Comparator;
import java.util.TreeMap;

record Product(Integer id, String name) implements Comparable<Product>
{
    @Override
    public int compareTo(Product p2)
    {
        return Integer.compare(this.id(), p2.id());
    }

}
public class TreeMapDemo3
{
    public static void main(String[] args)
    {
        TreeMap<Product, String> map = new TreeMap<>();
        map.put(new Product(222, "Mobile"), "Hyderabad");
        map.put(new Product(111, "Laptop"), "Chennai");
        map.put(new Product(333, "Camera"), "Pune");

      map.forEach((k,v)-> IO.println(k+" : "+v));

    }

}
```

```
package com.ravi.tree_map_demo;

import java.util.TreeMap;

record Student(Integer id, String name)
{
}
public class TreeMapDemo4
{
    public static void main(String[] args)
    {

        TreeMap<Student,String> map = new TreeMap<>((s1,s2)-> s1.id().compareTo(s2.id()));
        map.put(new Student(222, "Satish"), "Hyd");
        map.put(new Student(111, "Zuber"), "Bglr");

        map.forEach((k,v) -> System.out.println(k+" "+v));



    }

}
```

Hashtable<K,V>

----

public class Hashtable<K,V> extends Dictionary<K,V> implements Map<K,V>, Cloneable, Serializable.

* Legacy class [JDK 1.0]
* Methods are synchronized
* Performnace is not good
* Does not accept anything as a null [NPE]
* Default capacity is 11, load factor 0.75

It is predefined class available in java.util package under Map interface from JDK 1.0.

Like Vector, Hashtable is also form the birth of java so called legacy class.

It is the sub class of Dictionary class which is an abstract class.

**The major difference between HashMap(1.2) and Hashtable(1.0) is, HashMap methods are not synchronized where as Hastable methods are synchronized.

HashMap can accept one null key and multiple null values where as Hashtable does not contain anything as a null(key and value both). if we try to add null then JVM will throw an exception i.e NullPointerException.

The initial default capacity of Hashtable class is 11 where as loadFactor is 0.75.

It has also same constructor as we have in HashMap.(4 constructors)

1) Hashtable hs1 = new Hashtable();
   It will create the Hashtable Object with default capacity as 11 as well as load factor is 0.75

2) Hashtable hs2 = new Hashtable(int initialCapacity);
   will create the Hashtable object with user specified capacity

3) Hashtable hs3 = new Hashtable(int initialCapacity, float loadFactor);
   we can specify our own initialCapacity and loadFactor

4) Hashtable hs = new Hashtable(Map map);
   Interconversion of Map Collection

//Program :

```
import java.util.*;
public class HashtableDemo
     {
      public static void main(String args[])
          {
           Hashtable<Integer,String> map = new Hashtable<>();
           map.put(1, "Java");
           map.put(2, "is");
           map.put(3, "best");
           map.put(4,"language");

           //map.put(5,null);  //NPE

           IO.println(map);

           IO.println("......................");

            for(Map.Entry<Integer,String> m : map.entrySet())
              {
                   IO.println(m.getKey()+" = "+m.getValue());
             }
      }
}
```

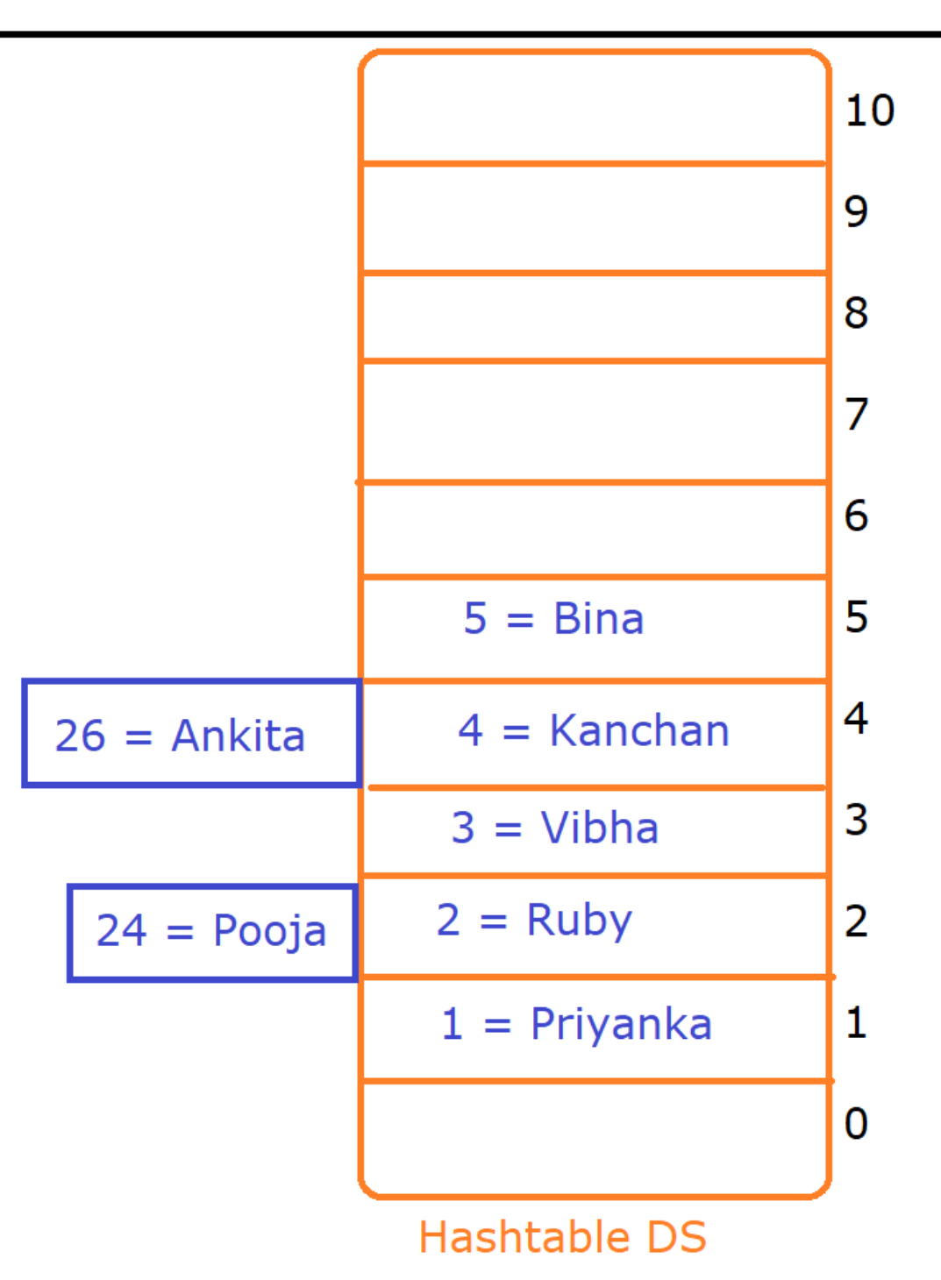
How Hashtable<K,V> works internally ?

```
 Hashtable<Integer,String> map = new Hashtable<>();
     map.put(1,"Priyanka");
     map.put(2,"Ruby");
     map.put(3,"Vibha");
     map.put(4,"Kanchan");

       map.putIfAbsent(5,"Bina");
       map.putIfAbsent(24,"Pooja");
       map.putIfAbsent(26,"Ankita");

     map.putIfAbsent(1,"Sneha");
     IO.println("Updated Map: "+map);
```

Bucket Index = key % table length

1 % 11 = 1

24 % 11 = 2



Output is :

Top to Bottom & left to right :

{5= Bina, 26=Ankita, 4= kanchan, 3= vibha, 24 = pooja, Ruby...Priyanka}

//Program:

```
import java.util.*;
public class HashtableDemo1
{
  public static void main(String args[])
     {
    Hashtable<Integer,String> map = new Hashtable<>();
     map.put(1,"Priyanka");
     map.put(2,"Ruby");
     map.put(3,"Vibha");
     map.put(4,"Kanchan");

       map.putIfAbsent(5,"Bina");
       map.putIfAbsent(24,"Pooja");
       map.putIfAbsent(26,"Ankita");

     map.putIfAbsent(1,"Sneha");
     IO.println("Updated Map: "+map);
 }
}
```

****WeakHashMap<K,V> [keys are weak type]**

----

public class WeakHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>

It is a predefined class in java.util package under Map interface.It was introduced from JDK 1.2V onwards.

While working with HashMap, keys of HashMap are of strong reference type. This means the entry of map will not be deleted by the garbage collector even though the custom key is set to be null as well as Object is also not eligible for Garbage Collector.

On the other hand while working with WeakHashMap, keys of WeakHashMap are of weak reference type. This means the entry and corresponding object of a map is deleted by the garbage collector if the key value is set to be null/assigning a new object because it is of weak type.

So, HashMap dominates over Garbage Collector whereas Garbage Collector dominates over WeakHashMap.

It does not implements Cloneable and Serailizable because It is mainly used for inventory system where we need to manage the data and we can insert and delete object data frequently in the inventory.

It contains 4 types of Constructor :
----

1) WeakHashMap wm1 = new WeakHashMap();

   Creates an empty WeakHashMap object with default capacity is 16 and load fator 0.75.

2) WeakHashMap wm2 = new WeakHashMap(int initialCapacity);

3) WeakHashMap wm3 = new WeakHashMap(int initialCapacity, float loadFactor);
   Eg:- WeakHashMap wm = new WeakHashMap(10,0.9);

   capacity - The capacity of this map is 10. Meaning, it can store 10 entries.

   loadFactor - The load factor of this map is 0.9. This means whenever our hashtable is filled up by 90%, the entries are moved to a new hashtable of double the size of the original hashtable.

4) WeakHashMap wm4 = new WeakHashMap(Map map);

```
package com.ravi.weak_hash_map;

import java.util.WeakHashMap;

record Product(Integer id, String name)
{
    @Override
    public void finalize()
    {
        IO.println("Product object is eligible for GC");
    }
}

public class WeakHashMapDemo
{
    public static void main(String[] args)
    {
       Product p1 = new Product(111, "Camera");

       WeakHashMap<Product,String> map = new WeakHashMap<>();
       map.put(p1, "Hyderabad");
       IO.println(map);

       p1 = null;

       System.gc(); //calling GC

       try
       {
       Thread.sleep(5000);
       }
       catch(InterruptedException e)
       {
           e.printStackTrace();
       }

       IO.println(map); //{}
    }



}
```

How to generate OR find out System hashcode value :

System class has provided a predefined native and static method called identityHashCode(Object obj), It is used to generate System hashcode.
It accepts Object as a parameter and return type of this method is int.

It is used to return default hash code of an object based on its memory identity, ignoring any overridden hashCode() method. [Object class hashcode]

public static native int identityHashCode(Object obj)

```
package com.ravi.identity_hash_map;

import java.util.Objects;

class Student
{
    String name;

    Student(String name) {
        this.name = name;
    }

    @Override
    public int hashCode() {
        return Objects.hash(name);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Student other = (Student) obj;
        return Objects.equals(name, other.name);
    }
}

public class SystemHashCode
{
    public static void main(String[] args)
    {
        Student s1 = new Student("Ravi");

        System.out.println(s1.hashCode()); // 100

        System.out.println(System.identityHashCode(s1)); // different

    }
}
```

IdentityHashMap<K,V>

----

public class IdentityHashMap<K,V> extends AbstractMap<K,V> implements Map<K,V>, Cloneable, Serializable

* JDK 1.4V
* Compare the keys using == operator only
* It does not use hashCode() & equals() method
* **Default capacity 32**
* To search the index position on the System array It uses
  System.identityHashCode() method
* It contains 3 constructor, load factor constructor is not available

It was introduced from JDK 1.4 onwards.

The IdentityHashMap uses == operator to compare key object.

HashMap uses hashCode() & equals() method.

We should use IdentityHashMap where we need to compare the keys by using reference or memory address instead of logical equality.

HashMap uses hashCode of the "Object key" to find out the bucket loaction in Hashtable, on the other hand IdentityHashMap does not use hashCode() method actually It uses System.identityHashCode(Object o)

IdentityHashMap is more faster than HashMap in case of key Comparison.

The default capacity is 32.

It has three constrcutors, It does not contain loadFactor specific constructor
bacause It uses flat array concept.

1) IdentityHashMap ihm1 = new IdentityHashMap();
  Will create IdentityHashMap with default capacity is 32 and fixed load factor is : capacity * 2/3;

2) IdentityHashMap ihm2 = new IdentityHashMap(int initialCapacity);
  Will create IdentityHashMap with user specified capacity

3) IdentityHashMap ihm3 = new IdentityHashMap(Map map)

```
package com.ravi.identity_hash_map;

import java.util.HashMap;
import java.util.IdentityHashMap;

public class IdentityHashMapDemo1
{
    public static void main(String[] args)
    {
       HashMap<String, Integer> map = new HashMap<>();
       map.put("Java", 1);
       map.put(new String("Java"), 2);
       IO.println(map);
       IO.println(map.size());  //1

       IO.println(".....................");

       IdentityHashMap<String, Integer> imap = new IdentityHashMap<>();
       imap.put("Java", 1);
       imap.put(new String("Java"), 2);
       IO.println(imap);
       IO.println(imap.size()); //2

    }

}
```